

AskData V2.1 —自然语言转 SQL 智能平台

完整技术文档 · 2026-07-23

AskData Team

2026-07-23

目录

1	文档摘要	3
2	项目概述	4
2.1	核心能力	4
2.2	规模概览	4
3	技术栈	5
3.1	后端	5
3.2	前端	5
3.3	外部服务	5
4	项目结构	6
5	V2 管道架构	8
5.1	整体流程	8
5.2	SQL Agent 核心架构	8
5.3	Agent 执行循环	9
5.4	Ambiguity 判断逻辑	9
5.5	Retrieval / RAG 在 Agent 中的角色	10
5.6	Prompt Context 结构	10
5.7	关键数据结构	10
5.7.1	IntentContract (意图契约)	10
5.7.2	QualityReport (质量报告)	11
5.7.3	判别响应 (Discriminated Response)	11
6	混合检索 RAG 架构	12
6.1	三层架构	12
6.2	Chunk 类型与索引	12
6.3	索引统计 (1867 条 chunk)	12
6.4	降级策略	13
7	API 端点	14
7.1	REST API	14
7.1.1	POST /api/query —执行查询	14
7.1.2	POST /api/query/stream —SSE 流式端点	15

7.1.3 会话管理

15

8 会话持久化

16

8.1 应用数据库 (SQLite)

16

8.2 SessionStore API

16

9 图表系统

18

9.1 确定性 ChartSpec

18

9.2 选择策略 (优先级从高到低)

18

10 歧义与答案可行性

19

10.1 材料歧义检测

19

10.1.1 状态机

19

10.1.2 澄清条件 (四重 AND)

19

10.1.3 词干化

19

11 SQL 质量门控

20

11.1 静态检查 (EvaluateStaticSql)

20

11.2 结果检查 (EvaluateResult)

20

11.3 候选选择 (CandidateLedger.SelectBest)

20

12 配置参考

21

13 CLI 命令

22

14 11 个 BIRD 数据库

23

15 关键设计原则

24

16 版本路线图

25

1 文档摘要

AskData V2.1 是一个面向 BIRD Mini-Dev 的自然语言转 SQL 平台。它的核心不是单次 prompt，而是一条可审计的分阶段管道：先判断问题是否可答，再检索 schema，上下文进入 ReAct SQL 生成，之后由静态 SQL 质量门控、执行结果验证和候选选择共同决定最终答案。

维度	V2.1 结果
产品能力	中英文提问、表格结果、图表规格、trace、SSE 流式响应、会话持久化
Text2SQL 安全性	SELECT-only、AST 校验、候选不可变、缺失实体不生成代理 SQL
检索能力	词法检索为稳定底座，可选 BGE-M3 + Milvus 混合检索
工程验证	后端 335+ 测试、前端测试与构建、V2 演示回归套件

本文件适合用于技术评审。若用于展示或答辩，可进一步拆分为一份 8-12 页的视觉版 slide deck。

2 项目概述

AskData V2.1 是一个**自然语言转 SQL (NL2SQL) 智能平台**，用户用中文或英文提出业务问题，系统自动生成只读 SQL、执行查询、返回结构化结果（含文字答案、数据表、图表建议、操作追踪）。平台以 BIRD 基准数据集为核心测试套件，支持 SQLite 数据库，并提供可选的企业级混合检索（RAG）能力。

2.1 核心能力

- **自然语言理解**：支持中英文问题，自动识别歧义并澄清
- **可信 SQL 生成**：6 次执行预算的分阶段管道，静态质量门控 + 结果验证
- **混合检索 RAG**：词法匹配 + BGE-M3 稠密向量 + Milvus 向量库，语义召回业务概念
- **会话持久化**：SQLite 存储会话和对话轮次，服务重启不丢失
- **运营追踪**：结构化 trace 事件 + SSE 流式端点
- **确定性图表**：规则引擎生成图表规格，前端 ECharts 渲染，LLM 不参与图表生成
- **答案可行性门控**：缺失实体不生成代理 SQL，歧义指标主动要求澄清
- **BIRD 评测**：严格/宽松执行精度评测 + 演示回归套件

2.2 规模概览

指标	数量
后端 Python 代码	8,563 行 (31 个模块)
测试代码	7,546+ 行 (335+ 个测试)
前端 TypeScript/React	4,843 行
Milvus 索引 chunk	1,867 条 (11 个数据库)
支持数据库数	11 个 (BIRD Mini-Dev)
API 端点	8 个 (含流式端点)

3 技术栈

3.1 后端

组件	技术	用途
Web 框架	FastAPI 0.111+	REST API + SSE 流式
ASGI 服务器	Uvicorn	生产级异步服务
数据模型	Pydantic 2.7+	请求/响应验证, 判别联合类型
配置管理	pydantic-settings	环境变量驱动配置
LLM 客户端	OpenAI SDK 1.34+	DeepSeek Chat (Chat Completions API)
SQL 解析	sqlglot 25.5+	AST 级别 SQL 验证
数据库引擎	SQLAlchemy 2.0+ / aiossqlite	SQLite 异步写入 + 查询执行
CLI 框架	Typer	命令行工具
嵌入模型	BGE-M3 (1024 维)	schema/value/evidence 稠密向量编码
向量库	Milvus (pymilvus 2.4+)	语义相似度搜索
包管理	uv	Python 依赖和虚拟环境
测试框架	pytest + pytest-asyncio	单元测试 + 集成测试

3.2 前端

组件	技术
框架	React 18 + TypeScript
构建工具	Vite
UI 组件库	Ant Design
图表	ECharts (从 ChartSpec 适配渲染)
状态管理	Zustand
SSE 解析	自定义增量解析器 (fetch + ReadableStream)
测试	Vitest + React Testing Library

3.3 外部服务

服务	地址	用途
LLM API	api.deepseek.com	DeepSeek Chat 模型
Embedding 服务	7.59.11.153:9106/v1	BGE-M3 OpenAI 兼容 API
Milvus	7.59.11.153:19530	向量相似度搜索

4 项目结构

```

askdata-v1/
├── backend/askdata/
│   ├── agent/
│   │   ├── graph.py          # 核心 AI 管道
│   │   ├── pipeline.py       # AgentGraph 顶层编排 (206 行)
│   │   ├── react_sql_agent.py # StagedSqlPipeline 6 阶段管道 (698 行)
│   │   ├── ambiguity.py      # ReActSqlAgent 工具调用 SQL 生成 (368 行)
│   │   ├── sql_quality.py    # AmbiguityGate 歧义检测与澄清 (387 行)
│   │   ├── intent.py         # 静态/结果质量门控 (1006 行)
│   │   ├── prompts.py        # IntentContract 意图数据结构
│   │   ├── state.py          # SQL 生成提示词模板 (241 行)
│   │   └── nodes/            # AgentState TypedDict
│   ├── api/                  # LangGraph 节点 (预留)
│   │   ├── app.py            # FastAPI 层
│   │   ├── routes.py         # 应用工厂 + 生命周期
│   │   ├── query_service.py   # REST 端点 (340 行)
│   │   ├── schemas.py        # QueryService 共享查询编排 (706 行)
│   │   ├── response_models.py # 请求模型 (46 行)
│   │   ├── session_store.py   # V2 判别响应模型 (89 行)
│   │   ├── session_manager.py # SQLite 会话持久化 (384 行)
│   │   └── trace.py           # 旧版内存会话管理器
│   ├── core/                 # 请求级别 trace 记录
│   │   ├── config.py         # 基础设施
│   │   ├── llm.py            # Settings (pydantic-settings)
│   │   └── paths.py          # LLMClient (49 行)
│   ├── db/                   # 项目根路径解析
│   │   ├── executor.py       # 数据库层
│   │   ├── validator.py      # SQLExecutor 分页 + 类型推断 (327 行)
│   │   └── query_runner.py    # SQLValidator sqlglot 只读验证
│   ├── tools/                # Execute 轻量 SQLite 执行器 (58 行)
│   │   ├── retriever.py      # 检索、分析、图表工具
│   │   ├── hybrid_retriever.py # BirdSchemaIndex + SemanticRetriever (493 行)
│   │   ├── embedding_client.py # HybridRetriever 混合检索融合 (242 行)
│   │   ├── vector_store.py   # EmbeddingClient 严格嵌入验证 (88 行)
│   │   ├── chart_builder.py  # MilvusVectorStore 懒加载封装 (170 行)
│   │   ├── analyzer.py       # ChartBuilder 确定性图表选择 (167 行)
│   │   └── skill_loader.py    # ResultAnalyzer LLM 中文解释
│   ├── eval/                 # SkillLoader 可复用 SQL 模板
│   │   ├── runner.py         # 评测
│   │   ├── metrics.py        # EvalRunner BIRD 评测 (242 行)
│   │   └── demo_suite.py     # BirdResultComparer 4 级匹配 (194 行)
│   ├── data/                 # DemoSuite V2 演示回归套件 (392 行)
│   │   └── bird_io.py         # BIRD 数据契约 + 遗留兼容 (251 行)
│   ├── skills/               # 技能 markdown 文件
│   │   ├── compare-periods.md
│   │   ├── ratio-analysis.md
│   │   └── rank-top-bottom.md
│   └── cli.py                # CLI 入口 (327 行)

```

```

├── frontend/
│   └── src/
│       ├── api/
│       │   ├── query.ts          # 后端 API 客户端
│       │   └── queryStream.ts    # SSE event-stream 解析器
│       ├── components/
│       │   ├── AppSidebar.tsx    # 56px 导航栏 + 数据库/历史抽屉
│       │   ├── ConversationDrawer.tsx # 持久化会话历史
│       │   ├── ClarificationPrompt.tsx # 行内歧义澄清
│       │   ├── ChartPanel.tsx     # ChartSpec → ECharts 适配
│       │   ├── AgentTrace.tsx     # 运营 trace 展示
│       │   ├── QueryResultView.tsx # 判别响应渲染
│       │   └── QueryInput.tsx     # 问题输入框
│       ├── pages/
│       │   └── QueryResultDemo.tsx # 主页面
│       ├── store/
│       │   └── queryStore.ts      # Zustand 状态管理
│       └── types/
│           └── query.ts           # TypeScript 类型定义
├── tests/                          # 后端测试 (7,546 行)
│   ├── test_sql_quality.py        # 质量门控测试 (915 行)
│   ├── test_query_service.py      # 查询服务测试 (702 行)
│   ├── test_pipeline.py           # 管道测试 (607 行)
│   ├── test_session_store.py      # 会话持久化测试
│   ├── test_response_models.py    # 响应模型测试
│   ├── test_ambiguity.py          # 歧义检测测试
│   ├── test_hybrid_retriever.py   # 混合检索测试
│   ├── test_embedding_client.py   # 嵌入客户端测试
│   ├── test_chart_builder.py      # 图表构建测试
│   ├── test_query_stream.py       # SSE 流式测试
│   ├── test_demo_suite.py         # 演示套件测试
│   └── fixtures/                  # 测试数据
│       └── v2_demo_cases.json     # V2 演示用例
├── benchmarks/                    # 版本化问题清单
├── data-processing/               # 团队 BIRD 数据准备管道
├── data/bird/                     # BIRD 数据集 (gitignored)
│   ├── databases/                 # 11 个 SQLite 数据库
│   └── processed/                 # 处理后的 schema/questions
├── docs/superpowers/
│   ├── specs/askdata-v2-design.md # V2 设计规范 (22K 字)
│   └── plans/askdata-v2-1-implementation.md # 15 任务实现计划
├── reports/                       # 评测报告 (gitignored)
├── scripts/
│   └── setup-dev-env.sh           # macOS 开发环境初始化
└── pyproject.toml

```

5 V2 管道架构

5.1 整体流程

阶段	核心组件	关键控制
入口与会话 可答性检查	POST /api/query, SessionStore AmbiguityGate.Check()	自动创建/恢复会话，注入历史上下文 clear / ambiguous / unanswerable 三态分流
Schema 检索	SemanticRetriever.Retrieve()	lexical 底座, RAG 可用时追加 semantic chunks
SQL 候选生成	ReActSqlAgent.GenerateCandidates()	tool-calling 生成候选，不直接决定最终答案
静态质量门控	EvaluateStaticSql()	SELECT-only、schema grounding、 join、aggregation、order、limit
执行与结果验证	Execute(), EvaluateResult()	只读执行；验证空结果、count、ranking、 ratio、输出列
候选选择	CandidateLedger.SelectBest()	coverage 优先，失败/警告更少优先
响应生成	ResultAnalyzer, ChartBuilder	answer/table/chart/SQL 全部来自同一最终候选

恢复预算按固定顺序推进：

```

initial
-> targeted_repair_1
-> targeted_repair_2
-> retrieval_expansion
-> alternate_plan
-> final_candidate

```

5.2 SQL Agent 核心架构

AskData V2.1 的 SQL Agent 不是“LLM 一次性输出 SQL”的结构，而是一个**确定性控制器 + ReAct 候选生成器 + SQL 质量门控 + 候选账本**组成的分层 agent。LLM 负责语义解释、SQL 草案和答案表述；是否可答、是否安全、是否符合问题形状、最终选择哪条 SQL，由确定性代码控制。

层	组件	决策边界
Orchestration	QueryService / AgentGraph	API、会话、trace 和响应类型由代码控制
Ambiguity	AmbiguityGate + StructuredInterpreter	LLM 只提交候选解释；代码判断是否需要澄清
Retrieval	SemanticRetriever	schema/evidence/value/RAG context 由检索层构造
Generation	ReActSqlAgent	LLM 只生成 SQL candidate，不选择最终答案
Validation	EvaluateStaticSql / EvaluateResult	安全、schema grounding、答案形状由 确定性规则判断
Recovery	StagedSqlPipeline	根据失败类型决定修复、扩检索、替代计划 或终止

层	组件	决策边界
Selection	CandidateLedger	最终 SQL 从候选账本中选择, answer/table/chart 同源

核心设计原则是：**ReAct** 只负责提出候选，**pipeline** 才是决策者。这样可以保留 LLM 的灵活性，同时避免 LLM 在错误结果上强行解释。

5.3 Agent 执行循环

每次查询最多消耗 6 次 SQL 执行预算。每个 candidate 都必须经过同一组检查：

1. **Generate**: `ReActSqlAgent.GenerateCandidates()` 读取 question、schema prompt、session context, 通过 tool-calling 生成 `SqlCandidateDraft`。
2. **Static gate**: `EvaluateStaticSql()` 对 SQL 做 AST 解析、安全检查、表/列校验、join 连通性和 aggregation/order/limit 检查, 输出 `QualityReport`。
3. **Execute**: `query_runner.Execute()` 只读执行通过静态检查或可执行的 SQL, 保留模型显式 LIMIT, 返回 columns + rows。
4. **Result gate**: `EvaluateResult()` 检查 scalar/list/ranking/ratio/grouped 的结果形状, 识别空结果、可疑 count 和排序方向错误。
5. **Ledger**: `CandidateLedger` 记录候选 SQL、质量报告、失败码、警告、执行结果和生成顺序。
6. **Decide**: `StagedSqlPipeline` 根据 ledger 和失败类型决定 targeted repair、retrieval expansion、alternate plan 或 final candidate。

失败类型直接决定恢复动作：

失败类型	典型原因	恢复动作
<code>syntax_or_safety</code>	SQL 语法错误、非 SELECT、多语句、SELECT *	要求 ReAct 生成修复候选, 不扩检索
<code>schema_grounding</code>	表/列不存在、join 条件错误	扩展 retrieval context 后重新生成
<code>answer_shape</code>	问题要 count/list/ranking, 但 SQL 形状不匹配	注入具体失败码, 做 targeted repair
<code>empty_or_suspicious</code>	空结果、可疑 count、排序方向不符	尝试 alternate plan
<code>repeated_no_progress</code>	多轮 SQL 等价或失败无变化	提前终止并返回 error/partial

5.4 Ambiguity 判断逻辑

AskData 的 ambiguity 不是“模型觉得不确定就问用户”。V2.1 使用**材料歧义**标准：只有当不同解释都会改变最终 SQL 且都能被 schema/evidence 支撑时，才向用户澄清。

判断流程：

1. `StructuredInterpreter` 让 LLM 提交最多 5 个结构化解释: entities、metrics、filters、supported_by。
2. `_IsSupported()` 用 schema/evidence 检查每个解释是否有数据库支撑。
3. `_MaterialSignature()` 将解释压缩成会影响 SQL 的 signature: 实体、指标、过滤、聚合、分组、时间、排序。
4. 如果所有有效解释的 material signature 相同, 说明歧义不影响 SQL, 直接继续。
5. 如果多个有效解释会生成不同 SQL, 且上下文/默认规则无法确定一个, 返回 `ClarificationResponse`。
6. 如果没有任何解释被 schema 支撑, 返回 `unanswerable`, 不生成代理 SQL。

输入情况	输出	原因
“show me the data”	clarification	缺少目标实体/表，多个解释会改变 SQL
“top 4 leagues had most games”	clear	leagues 可词干化到 League, games 可由 retrieval 指向 Match
“free meals ages 15-17” 但 schema 只有 ages 5-17	unanswerable 或带限制说明	请求指标不存在，不能用近似字段冒充
“revenue” 同时存在 gross/net 且都可答	clarification	指标选择会改变 SQL

5.5 Retrieval / RAG 在 Agent 中的角色

检索层的目标不是替 LLM 作答，而是把**正确表、列、join 路径、证据、值映射**放进 prompt，让 SQL candidate 更容易 grounded。

来源	产生内容	解决的问题
lexical schema match	表名、列名、PK/FK、邻居表	精确词命中、join backbone
BIRD evidence	指标定义、别名、业务说明	“decrease rate” 等公式类问题
value chunks	低基数枚举、代码值、日期/数值范围	用户字面值和数据库存储值不一致
dense vector chunks	语义相近的 schema/value/evidence/example	games→Match、leagues→League 等词汇差异
session context	上轮 SQL、澄清选择、修复反馈	多轮追问和 targeted repair

混合检索采取 “lexical-first + dense optional” 的策略。lexical 是稳定底座；Milvus/embedding 可用时再加入 dense chunks，通过 Reciprocal Rank Fusion 合并；服务不可用时 trace 里记录安全 warning，并继续 lexical retrieval。

5.6 Prompt Context 结构

进入 ReAct 的 prompt 由四类信息组成：

prompt 区块	内容	约束
system instructions	SQL 安全、列选择纪律、aggregation/join/ranking/ratio 规则	长期稳定规则
schema prompt	相关表、列、主键、外键、evidence、business instructions	只允许使用其中的表和列
question analysis context	deterministic intent、requested outputs、value links	JSON fenced block, 标记为 data context
recovery feedback	上一轮 SQL、失败码、修复目标	只在 targeted repair 阶段注入

这一结构避免把检索结果、用户问题、修复指令混在一起，降低 prompt injection 和 “凭空补 schema” 的风险。

5.7 关键数据结构

5.7.1 IntentContract (意图契约)

```
class IntentContract(BaseModel):
    shape: Literal["scalar", "listing", "ranking", "ratio", "grouped", "comparison"]
```

```

entities: list[str]          # 涉及的实体 (表/概念)
output_attributes: list[str] # 期望输出字段
metrics: list[str]           # 度量指标 (count, average, ratio...)
filters: list[str]           # 过滤条件
grouping: list[str]          # 分组字段
order: Literal["ascending", "descending"] | None
expected_max_rows: int | None
time_condition: str | None

```

5.7.2 QualityReport (质量报告)

```

class QualityReport(BaseModel):
    passed: bool
    failures: list[str]          # 阻塞性失败码
    warnings: list[str]         # 非阻塞性警告
    coverage: float              # 意图覆盖比例 0-1
    covered_elements: list[str] # 已覆盖的语义元素
    directness: float           # SQL 简洁度
    semantic_outputs: dict      # 每列的语义标注

```

5.7.3 判别响应 (Discriminated Response)

```

QueryResponse = Annotated[
    AnswerResponse | ClarificationResponse | PartialResponse | ErrorResponse,
    Field(discriminator="kind"),
]

```

```

# AnswerResponse(kind="answer")
#   answer, sql, columns, rows, chart, confidence, trace

# ClarificationResponse(kind="clarification")
#   clarification_id, question, options, recommended_option_id

# PartialResponse(kind="partial")
#   answer, limitations, suggestions, confidence

# ErrorResponse(kind="error")
#   code, message, retryable, suggestions

```

6 混合检索 RAG 架构

6.1 三层架构

层级	组件	做什么	可靠性设计
Layer 1	BirdSchemaIndex	问题 token 与表名、列名、描述做交集匹配	外键邻居扩展；无匹配时回退到前 8 张表
Layer 2	HybridRetriever	对问题文本做 BGE-M3 embedding, 并在 Milvus 中按 database_id 搜索	自动分批、长文本截断、服务异常捕获
Layer 3	ReciprocalRankFusion	融合 lexical、schema、value、evidence、example 排名	source_type 分组、join neighbor 扩展、coverage 检查

混合检索的数据流：

question

- > lexical candidates
- > dense candidates from Milvus
- > reciprocal rank fusion
- > join-neighbor expansion
- > schema prompt + retrieval trace

当前实现是 coverage check + terminology expansion, 不是严格的 HyDE 流程。它不会先生成 hypothetical document 再 embedding。

6.2 Chunk 类型与索引

类型	内容	属性标识
schema	表名、列名、类型、主键、外键、join 邻居	source_type=schema, table_name, column_name
value	低基数分类值、代码映射、数值范围、null 比例	source_type=value
evidence	BIRD 证据文本、业务指令、别名、指标定义	source_type=evidence
example	验证过的 question→SQL 对, 标记为示例	source_type=example

每个 chunk 都有稳定标识符：{database_id}::{source_type}::{table_name}::{column_name}::{hash}

6.3 索引统计 (1867 条 chunk)

数据库	总计	schema	evidence	example
european_football_2	308	206	51	51
formula_1	239	107	66	66
card_games	225	121	52	52
codebase_community	177	79	49	49
thrombosis_prediction	167	67	50	50
california_schools	152	92	30	30
student_club	152	56	48	48

数据库	总计	schema	evidence	example
superhero	145	41	52	52
financial	123	63	30	30
toxicology	95	15	40	40
debit_card_specializing	84	26	28	30

6.4 降级策略

当 Embedding 服务或 Milvus 不可用时:

1. **启动时验证** → `embedding.Validate()` + `vector.Search(probe)` → 失败记入缓存
2. **运行时降级** → `HybridSchemaIndex(lexical, fallback_warning)` → 仅词法检索
3. **trace 警告** → `retrieval_trace: [{status: "warning", message: "Semantic retrieval unavailable"}]`
4. **不崩溃** → 任何向量服务异常被捕获, 不抛出到调用方

7 API 端点

7.1 REST API

7.1.1 POST /api/query — 执行查询

请求体:

```
{
  "question": "Which top 4 leagues had the most games in the 2015-2016 season?",
  "database_id": "european_football_2",
  "session_id": "optional-existing-session-id"
}
```

或歧义延续:

```
{
  "database_id": "european_football_2",
  "session_id": "session-id",
  "clarification": {
    "clarification_id": "c1",
    "option_id": "net"
  }
}
```

响应 (判别联合类型):

kind="answer":

```
{
  "kind": "answer",
  "session_id": "uuid",
  "turn_id": "uuid",
  "answer": " 自然语言回答",
  "sql": "SELECT ...",
  "columns": ["name"],
  "rows": [{"name": "Spain LIGA BBVA"}],
  "chart": {"type": "horizontal_bar", ...},
  "confidence": "high",
  "trace": [{"step": "RetrieveSchema", "status": "success", ...}]
}
```

kind="clarification":

```
{
  "kind": "clarification",
  "session_id": "uuid",
  "turn_id": "uuid",
  "clarification_id": "c1",
  "question": "Which interpretation should I use?",
  "options": [
    {"id": "gross", "label": "Gross revenue", "description": "..."},
    {"id": "net", "label": "Net revenue", "description": "..."}
  ],
  "recommended_option_id": "net"
}
```

```
}

kind="error":
{
  "kind": "error",
  "code": "query_failed",
  "message": " 查询失败，请稍后重试",
  "retryable": true,
  "suggestions": [" 重试", " 换一种问法"]
}

kind="partial":
{
  "kind": "partial",
  "answer": " 部分结果...",
  "limitations": [" 无法确定时间范围"],
  "suggestions": [" 请指定具体日期"],
  "confidence": "low"
}
```

7.1.2 POST /api/query/stream —SSE 流式端点

```
event: trace
data: {"step":"RetrieveSchema","status":"success","message":"Schema expanded.","sequence":1}

event: trace
data: {"step":"GenerateSql","status":"success","message":"SQL candidate
↳ generated.","sequence":2}

event: final
data: {"kind":"answer","session_id":"...","turn_id":"...","answer":"...","sql":"...",...}
```

支持的事件类型: trace、clarification、final、error。序列号单调递增。客户端断开触发尽力取消。

7.1.3 会话管理

方法	路径	说明
GET	/api/sessions?limit=50	列出最近会话
POST	/api/sessions	创建新会话
GET	/api/sessions/{id}	获取会话及有序轮次
DELETE	/api/sessions/{id}	删除会话（级联删除轮次）
GET	/api/metadata	列出可用数据库

8 会话持久化

8.1 应用数据库 (SQLite)

与业务数据库独立，存储位置：data/askdata-app.sqlite（可配置）

```
CREATE TABLE sessions (
    id TEXT PRIMARY KEY,
    database_id TEXT NOT NULL,
    title TEXT NOT NULL DEFAULT '',
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL
);

CREATE TABLE turns (
    id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    question TEXT NOT NULL,
    response_kind TEXT NOT NULL,
    answer TEXT,
    sql TEXT,
    result_preview_json TEXT,    -- 有界预览 (<=100 行)
    chart_json TEXT,
    confidence TEXT,
    error_json TEXT,
    trace_json TEXT NOT NULL DEFAULT '[]',
    created_at TEXT NOT NULL
);

CREATE TABLE clarifications (
    id TEXT PRIMARY KEY,
    turn_id TEXT NOT NULL UNIQUE REFERENCES turns(id) ON DELETE CASCADE,
    prompt TEXT NOT NULL,
    options_json TEXT NOT NULL,
    resolution_json TEXT,
    status TEXT NOT NULL,        -- pending | resolved
    created_at TEXT NOT NULL,
    resolved_at TEXT
);
```

数据库启用：PRAGMA foreign_keys=ON、PRAGMA journal_mode=WAL、PRAGMA busy_timeout=5000。

8.2 SessionStore API

```
class SessionStore:
    async def Initialize()          # 创建表 + PRAGMA
    async def Close()              # 关闭连接
    async def CreateSession(database_id, title='') -> session_id
    async def ListSessions(limit=50) -> list[SessionSummary]
    async def GetSession(session_id) -> SessionWithTurns | None
    async def DeleteSession(session_id) # 级联删除
    async def SaveTurn(session_id, turn) # 写入轮次
```

```
async def CreateClarification(turn_id, prompt, options)
async def ResolveClarification(session_id, clarification_id, resolution)
```

写操作通过 `asyncio.Lock` 串行化。ISO UTC 时间戳。JSON 序列化存储有界结果预览。

9 图表系统

9.1 确定性 ChartSpec

LLM 不参与图表生成。所有图表规格由确定性规则引擎产生：

```
class ChartSpec(BaseModel):
    type: Literal["line", "vertical_bar", "horizontal_bar", "pie", "scatter"]
    title: str
    category_field: str | None
    value_fields: list[str]
    category_label: str | None
    value_labels: dict[str, str]
    reason: Literal["time_series", "comparison", "ranking", "proportion", "correlation"]
```

9.2 选择策略（优先级从高到低）

1. 时间字段 + 数值指标 → **line**（折线图）
2. 明确的比例/百分比 + ≤ 6 个非负类别 → **pie**（饼图）
3. 两个数值度量 + ≥ 5 行 → **scatter**（散点图）
4. Ranking 问题（top/bottom/most/least） → **horizontal_bar**（水平柱状图）
5. 类别比较 → **vertical_bar**（垂直柱状图）
6. 稀疏/不适用 → **None**（纯表格）

前端 `ChartPanel.tsx` 将 `ChartSpec` 映射为 ECharts 配置。CSS 变量驱动的主题适配。aria-label 屏幕阅读器支持。

10 歧义与答案可行性

10.1 材料歧义检测

`AmbiguityGate.Check(question, schema, evidence, session_context) → AmbiguityDecision`

10.1.1 状态机

`StructuredInterpreter.Interpret()`

→ LLM 返回 interpretations (最多 5 个)

→ `_IsSupported()`: 每个 entity/metric/filter 必须在 schema 中有支撑

├ 无有效 interpretation → "unanswerable"

├ 1 个有效 interpretation:

├ 所有 interpretations 共同的 material signature

├ → "resolvable_from_context"

├ 唯一 interpretation → "clear"

├ 多个有效 interpretation:

├ 相同 material signature → "resolvable_from_context"

├ 上下文或默认值支持一个 → "resolvable_from_context"

├ 需要用户选择 → "materially_ambiguous"

10.1.2 澄清条件 (四重 AND)

澄清仅在以下全部满足时触发:

1. 至少两个由 schema/evidence 支持的解释
2. 上下文和默认值均无法确定主导解释
3. 选择会改变: 实体、指标、过滤、聚合、分组、时间范围、排序方向
4. 数据库可以回答两种解释

10.1.3 词干化

轻量级英语词干化处理单复数、动词变位:

```
_STEM_RE = re.compile(
    r"(ies|sses|ses|ches|shes|xes|zes|ves|uses|s|ing|ed|er|est|ment|ness|ly)$"
)
# "leagues" → "league", "students" → "student", "running" → "runn"
```

11 SQL 质量门控

11.1 静态检查 (EvaluateStaticSql)

基于 sqlglot AST 的确定性检查，不调用 LLM:

检查类别	失败码	条件
安全	unsafe_sql	非 SELECT / 多语句 / SELECT *
Schema	unknown_table	引用的表不在 schema 中
	unknown_column	引用的列不在 schema 中
	invalid_join_using	JOIN 使用了不存在的列
	unconnected_join	JOIN 没有连接条件
语义	missing_count_aggregation	需要 COUNT 但 SQL 中没有 (含 ORDER BY 检查)
	missing_output_attribute	请求的输出字段不在 SELECT 中
	missing_metric	请求的指标不在 SELECT 中
	missing_grouping	缺少 GROUP BY
	missing_order	Ranking 需要 ORDER BY
	wrong_order_direction	排序方向不匹配
	wrong_order_target	排序目标不是请求的指标
	missing_limit	Ranking 需要 LIMIT
	missing_entity	意图实体未在 SQL 中引用 (词干化匹配)

11.2 结果检查 (EvaluateResult)

执行后验证:

失败码	条件
empty_result	期望有数据但返回 0 行
missing_result_attribute	请求的输出字段不在结果列中
missing_result_metric	请求的指标不在结果列中 (ranking 豁免)
suspicious_count	COUNT 结果不是非负整数
ranking_order_mismatch	Ranking 排序方向与意图不匹配
inspection_query_result	检查查询被误认为最终答案

11.3 候选选择 (CandidateLedger.SelectBest)

优先级 (从高到低):

1. 平均覆盖率 (static + result) 最高
2. 失败数最少
3. 警告数最少
4. SQL 简洁度最高
5. 生成顺序最早 (同分时)

12 配置参考

```
# .env 文件
# =====

# LLM 配置 (DeepSeek 或任何 OpenAI 兼容 API)
LLM_API_BASE=https://api.deepseek.com
LLM_API_KEY=sk-xxxxxxx
LLM_MODEL_NAME=deepseek-chat

# BIRD 数据集路径
BIRD_DATA_DIR=data/bird

# 应用数据库 (会话存储)
APP_DATABASE_PATH=data/askdata-app.sqlite

# Embedding 配置
EMBEDDING_API_URL=http://7.59.11.153:9106/v1      # API base URL (不含 /embeddings)
EMBEDDING_API_KEY=dummy                          # OpenAI SDK 要求非空
EMBEDDING_MODEL=BAAI/bge-m3                      # 模型名
EMBEDDING_DIMENSION=1024                         # 向量维度

# Milvus 向量库
MILVUS_URI=http://7.59.11.153:19530               # 完整 URI
MILVUS_HOST=7.59.11.153                          # 或使用 HOST+PORT
MILVUS_PORT=19530
MILVUS_COLLECTION=askdata_schema_chunks

# 向量检索开关
VECTOR_RETRIEVAL_ENABLED=true

# Rerank (预留)
RERANK_API_URL=http://7.59.11.153:9107/rerank

# MySQL (V2.2 预留)
MYSQL_HOST=7.59.11.153
MYSQL_PORT=13306
MYSQL_USER=intern
MYSQL_PASSWORD=xxx
MYSQL_DATABASE=intern_db
```

13 CLI 命令

启动 API 服务器（热重载）

```
uv run askdata serve --reload
```

列出可用数据库

```
uv run askdata databases
```

为指定数据库构建向量索引

```
uv run askdata index-schema --database-id <db_id>
```

交互式聊天

```
uv run askdata chat --database-id <db_id>
```

BIRD 评测

```
uv run askdata eval-bird \  
  --model-name deepseek-v4-pro \  
  --question-manifest benchmarks/bird-minidev-v4pro-seed42-100.json \  
  --out reports/bird-eval-v4pro-100.json
```

V2 演示回归套件

```
uv run askdata eval-demo \  
  --cases tests/fixtures/v2_demo_cases.json \  
  --out reports/v2-demo.json
```

运行完整测试套件

```
uv run pytest -q # 后端 335 测试  
cd frontend && npm test -- --run && npm run build # 前端 72 测试 + 构建
```

14 11 个 BIRD 数据库

数据库	表数	大小	典型问题数
california_schools	3 (frpm, satscores, schools)	11 MB	30
card_games	6 (cards, foreign_data, legalities, sets, set_translations)	250 MB	52
codebase_community	8 (badges, comments, postHistory, postLinks, posts, ...)	459 MB	49
debit_card_specializing	5 (customers, gasstations, products, transactions_1k, ...)	33 MB	30
european_football_2	7 (Player, Player_Attributes, League, Country, Team, Team_Attributes, Match)	570 MB	51
financial	8 (account, card, client, disp, district, loan, order, trans)	68 MB	30
formula_1	13 (circuits, constructors, drivers, seasons, races, ...)	21 MB	66
student_club	8 (event, major, zip_code, attendance, budget, ...)	2.5 MB	48
superhero	10 (alignment, attribute, colour, gender, publisher, ...)	232 KB	52
thrombosis_prediction	3 (Examination, Patient, Laboratory)	7.0 MB	50
toxicology	4 (atom, bond, connected, molecule)	2.6 MB	40

15 关键设计原则

1. **LLM 只做语言和语义判断**：SQL 生成/修复、歧义解释、答案合成。其他一切由确定性代码控制。
2. **候选不可变性**：选定最终候选后，answer、chart、table、returned SQL 全部来自同一候选。
3. **不暴露思维链**：trace 只包含结构化操作事件，原始 LLM 输出不透露。
4. **缺失实体不代理**：请求的概念不在 schema 中时，返回 unanswerable 而非用错误实体生成 SQL。
5. **向量相似度不是意图**：低相似度本身不触发澄清，只当存在 schema 支撑的多个 plausible 解释时才澄清。
6. **图表不含可执行代码**：ChartSpec 只有声明性字段名称和值，无 JavaScript、颜色、任意 ECharts 选项。
7. **前端是真相来源**：V2 保留现有前端 shell、排版、token、响应式行为。

16 版本路线图

版本	范围	状态
V1	基础 NL2SQL: ReAct agent + 语法检索 + BIRD 评测	已发布
V2.1	可信 SQL、歧义澄清、混合检索、会话持久化、流式追踪、确定性图表	当前版本
V2.2	MySQL/PostgreSQL 适配器、跨编码器重排序、检索反馈学习	计划中
V2.3	认证授权、多用户、配额、审计、分布式工作器	远期